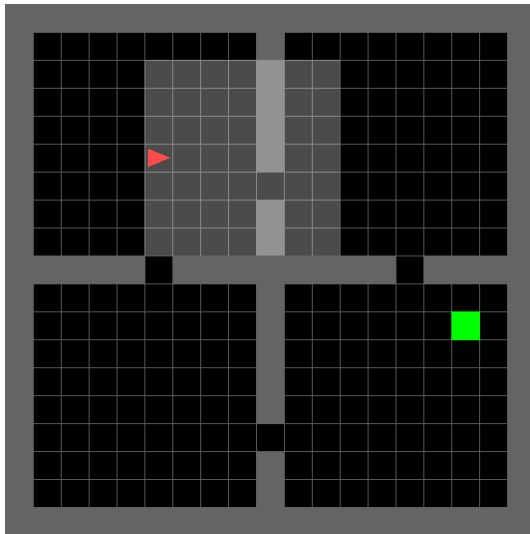


Hierarchical RL

Kuderov Petr

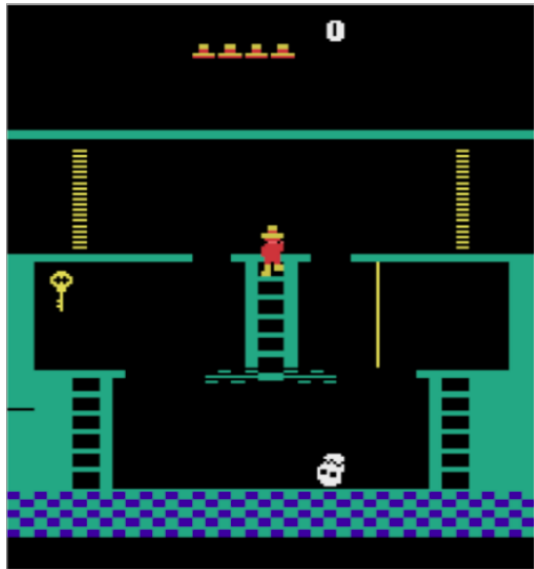
February 27, 2020

The world through the eyes of a RL agent



Common RL bottlenecks

- ▶ Sparse and long-term rewards
- ▶ Inefficient long-term planning
- ▶ Inefficient training
- ▶ Local minima



RL agents vs humans: go-to-the-shop example

- ▶ Humans use high-level abstractions to operate at **different scales**.
- ▶ RL agents operate at **one scale**, defined by their primitive actions.

Go to the shop

- ▶ go outside
- ▶ cross the road
- ▶ go along the street
- ▶ ...
 - ▶ walk
 - ▶ make a step
 - ▶ turn

Hierarchical Reinforcement Learning (HRL)

Make RL agents **discover** and **exploit** abstractions at different scales of state/action space and time.

The promise of HRL

- ▶ Long-term credit assignment and memory
faster learning, better generalization
- ▶ Structural exploration
explore with high-level actions, consistent exploration
- ▶ Transfer learning
different levels of hierarchy can encapsulate different reusable knowledge
- ▶ Explainable agents
understandable and predictable behaviour

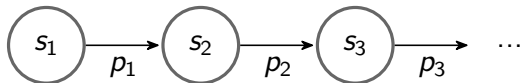
What is a high level action?

Intuition: high-level action is an abstraction, which encapsulates lower-level actions.

How to treat them?

- ▶ indivisible prolonged action or a sequence of lower-level actions?
- ▶ ... or both?

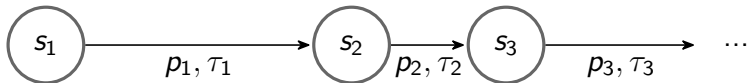
Markov process



Transition function is $P(s'|s)$

No notion of time

Semi-Markov process



Transition function becomes $P(s', t|s)$

From MDP to Semi-MDP (SMDP)

Markov Decision Process

- ▶ $\mathcal{S}$ — set of states
- ▶ $\mathcal{A}$ — set of actions
- ▶ $\mathcal{T}: \mathcal{S} \times \mathcal{A} \rightarrow P(s'|s, a)$ — transitions
- ▶ $\mathcal{R}$ — reward function

From MDP to Semi-MDP (SMDP)

Markov Decision Process

- ▶ $\mathcal{S}$ — set of states
- ▶ $\mathcal{A}$ — set of actions
- ▶ $\mathcal{T}: \mathcal{S} \times \mathcal{A} \rightarrow P(s'|s, a)$ — **transitions**
- ▶ $\mathcal{R}$ — reward function

Semi-Markov Decision Process

- ▶ $\mathcal{S}$ — set of states
- ▶ $\mathcal{A}$ — set of actions
- ▶ $\mathcal{T}: \mathcal{S} \times \mathcal{A} \rightarrow P(s', t|s, a)$ — **transitions**
- ▶ $\mathcal{R}$ — reward function

MDP vs SMDP

Time →

MDP



Discrete time
Homogeneous discount

SMDP



Continuous time, discrete events
Interval-dependent discount

SMDP. Bellman equations example

MDP:

$$\begin{aligned} Q^\pi(s, a) &= \mathbb{E}_{s' \sim P} [r(s, a) + \gamma V(s')] \\ &= r(s, a) + \sum_{s'} \gamma P(s'|s, a) V(s') \end{aligned}$$

SMDP:

$$\begin{aligned} Q^\pi(s, a) &= \mathbb{E}_{s', \tau \sim P} [R(s, a) + \gamma^\tau V(s')] \\ &= R(s, a) + \sum_{s'} \sum_{\tau} \gamma^\tau P(s', \tau | s, a) V(s') \\ &= R(s, a) + \sum_{s'} P(s'|s, a) V(s') \end{aligned}$$

, where $P(s'|s) = \sum_{\tau} \gamma^\tau P(s', \tau | s, a)$

SMDP. Q-learning

MDP. One-step Q-learning update on (s, a, r, s') sample:

$$TD_{tar} = r + \gamma \max_{a'} Q_k(s', a')$$
$$Q_{k+1}(s, a) = Q_k(s, a) + \alpha(TD_{tar} - Q_k(s, a))$$

SMDP. One-step Q-learning update on (s, a, r, s', τ) sample:

$$TD_{tar} = r + \gamma^\tau \max_{a'} Q_k(s', a')$$
$$Q_{k+1}(s, a) = Q_k(s, a) + \alpha(TD_{tar} - Q_k(s, a))$$

High-level actions

Solved: how to work with high-level actions as prolonged actions?

Yet to be solved: how to work with high-level actions as a sequence of lower-level actions?

Options Framework ¹

Answers to the question:

What is the minimal extension of the reinforcement learning framework that allows a general treatment of temporally abstract knowledge and action?

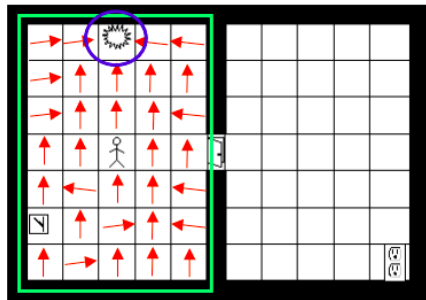
¹Between MDPs and semi-MDPs: A framework for temporal abstraction in reinforcement learning (1999)

Option as a generalization of an action

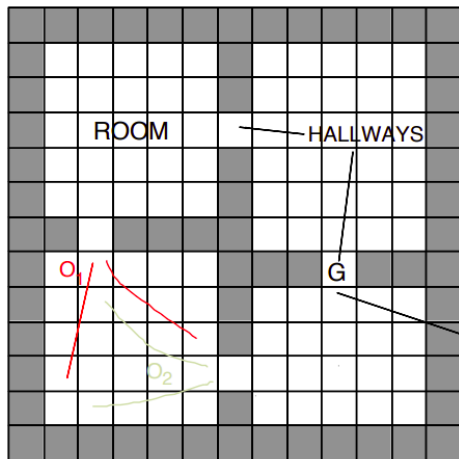
Defined as triple $\langle \mathcal{I}, \pi, \beta \rangle$:

- ▶ *Initiation set* $\mathcal{I}$ - states, where an option can be applied
- ▶ *Intra-option policy* π - policy, which defines behavior of an option
- ▶ *Termination policy* β - state-specific probability of an option being finished

Option: go to the couch.

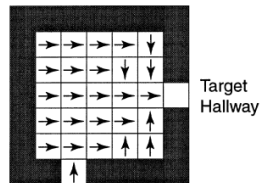


Four-room example



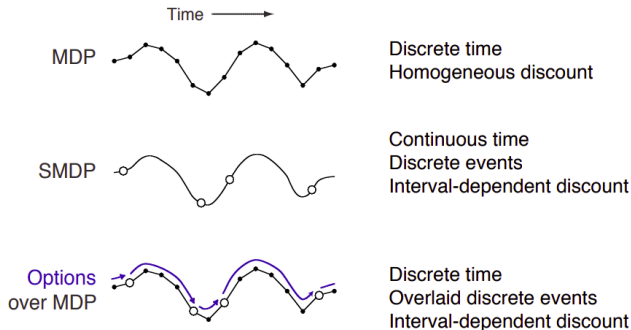
8 multi-step options,
to each room's 2
hallways

A hallway option.



Goal states are
given a terminal
value of 1

Policy over options training



How can we optimize policy over options?

- ▶ treat options as indivisible prolonged actions and use Q-learning for SMDP
- ▶ use the knowledge, that options are not indivisible, and improve Q-learning

Intra-option Q-learning

SMDP. One-step Q-learning update on (s, o, r, s', τ) sample:

$$TD_{tar} = r + \gamma^\tau \max_{o'} Q_k(s', o')$$
$$Q_{k+1}(s, o) = Q_k(s, o) + \alpha(TD_{tar} - Q_k(s, o))$$

Intra-option Q-learning

SMDP. One-step Q-learning update on (s, o, r, s', τ) sample:

$$TD_{tar} = r + \gamma^{\tau} \max_{o'} Q_k(s', o')$$
$$Q_{k+1}(s, o) = Q_k(s, o) + \alpha(TD_{tar} - Q_k(s, o))$$

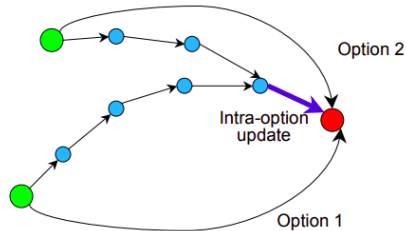
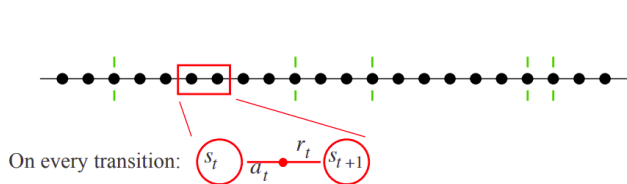
SMDP. One-step **intra-option** Q-learning update on (s, o, a, r, s', τ) sample:

$$TD_{tar} = r + \gamma U_k(s', o)$$
$$Q_{k+1}(s, o) = Q_k(s, o) + \alpha(TD_{tar} - Q_k(s, o))$$

, where $U(s, o)$ - state-option pair value *upon arrival*:

$$U(s, o) = (1 - \beta(s))Q(s, o) + \beta(s) \max_{o'} Q(s, o')$$

Intra-option Q-learning bonus

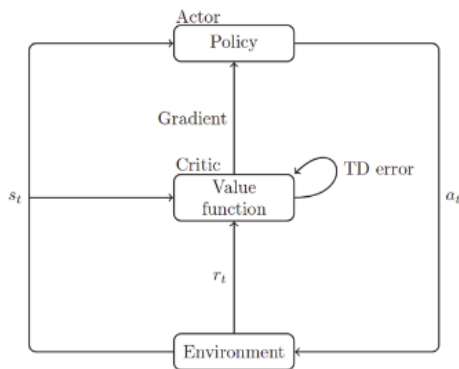


Update every option who can produce the same (s_t, a_t) step!

Open problem

Still don't know how to do both **discovering** and **learning** options

Actor-Critic recap

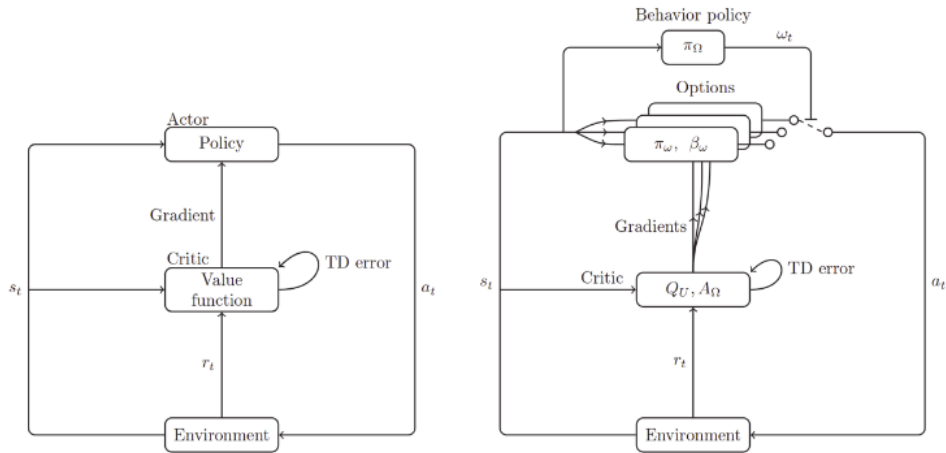


Actor optimizes policy
Critic estimates $Q(s, a)$

Policy gradient:

$$\nabla_{\Theta} J(\pi_{\Theta}) = \mathbb{E}[Q(s_t, a_t) \cdot \nabla_{\Theta} \log \pi_{\theta}(a_t | s_t)]$$

Option-Critic ²



²The Option-Critic Architecture (2016)

Training. Critic

Estimates $Q(s, o)$: intra-option Q-learning update on (s, o, a, r, s', τ) :

$$U(s, o) = (1 - \beta(s))Q(s, o) + \beta(s) \max_{o'} Q(s, o')$$

$$TD_{tar} = r + \gamma U_k(s', o)$$

$$Q_{k+1}(s, o) = Q_k(s, o) + \alpha(TD_{tar} - Q_k(s, o))$$

Estimates $Q(s, o, a)$: intra-option Q-learning update on (s, o, a, r, s', τ) :

$$Q_k(s, o) = \mathbb{E}_{a \sim P}[Q(s, o, a)] = \mathbb{E}_{a, s' \sim P}[r + \gamma U(s', o)]$$

$$TD_{tar} = r + \gamma U_k(s', o)$$

$$Q_{k+1}(s, o, a) = Q_k(s, o, a) + \alpha(TD_{tar} - Q_k(s, o, a))$$

.. or uses (s, o, a, r, s', τ) as sample estimates for $Q(s, o, a) = \mathbb{E}_{s' \sim P}[r + \gamma U(s', o)]$

Training. Actor

Has induced master-policy as ϵ -greedy softmax policy, based on $Q(s, o)$

Training. Actor

Learns intra-option policy with **Intra-Option Policy Gradient**:

$$\nabla_{\Theta} J(\Theta, \eta) = \mathbb{E}_{\substack{s, o \sim P \\ a \sim \pi_{o, \theta}}} [Q(s, o, a) \nabla_{\Theta} \log \pi_{o, \theta}(a|s)]$$

Intuition:

$$\begin{aligned} \nabla_{\Theta} J(\Theta, \eta) &= \nabla \mathbb{E}_{s, o \sim P} [Q(s, o)] = \nabla \mathbb{E}_{\substack{s, o \sim P \\ a \sim \pi_{o, \theta}}} [Q(s, o, a)] \\ &= \nabla \mathbb{E}_{s, o \sim P} \left[\sum_a \pi_{o, \theta}(a|s) Q(s, o, a) \right] \\ &= \mathbb{E}_{s, o \sim P} \left[\sum_a [\nabla \pi_{o, \theta}(a|s) Q(s, o, a) + \pi_{o, \theta}(a|s) \nabla Q(s, o, a)] \right] \\ &= \mathbb{E}_{s, o \sim P} \left[\sum_a \pi_{o, \theta}(a|s) Q(s, o, a) \nabla \log \pi_{o, \theta}(a|s) \right] \\ &= \mathbb{E}_{\substack{s, o \sim P \\ a \sim \pi_{o, \theta}}} [Q(s, o, a) \log \nabla \pi_{o, \theta}(a|s)] \end{aligned}$$

Training. Actor

Learns termination policy with **Termination Policy Gradient**:

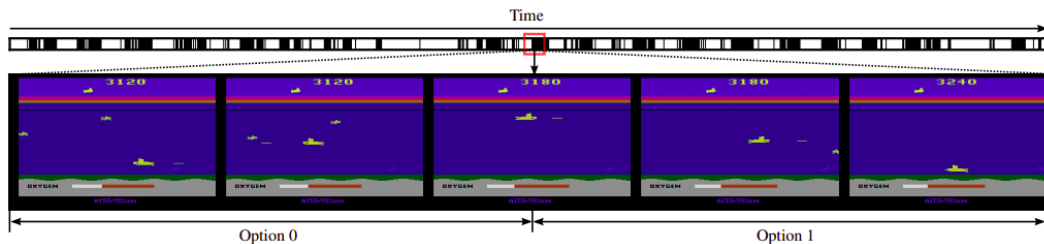
$$\nabla_{\eta} J(\Theta, \eta) = \mathbb{E}_{\substack{s', o \sim P \\ a \sim \pi_{o, \theta}}} [-A(s', o) \nabla_{\eta} \beta_{o, \eta}(s')]$$

Intuition:

$$\begin{aligned} U(s, o) &= (1 - \beta(s))Q(s, o) + \beta(s) \max_{o'} Q(s, o') \\ &= Q(s, o) + \beta(s)(V(s) - Q(s, o)) = Q(s, o) - \beta(s)A(s, o) \end{aligned}$$

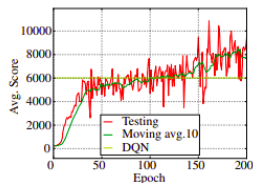
$$\begin{aligned} \nabla_{\eta} J(\Theta, \eta) &= \nabla \mathbb{E}_{s, o \sim P} [Q(s, o)] = \nabla \mathbb{E}_{\substack{s, o \sim P \\ a \sim \pi_{o, \theta}}} [Q(s, o, a)] \\ &= \nabla \mathbb{E}_{\substack{s, o, s' \sim P \\ a \sim \pi_{o, \theta}}} [r + \gamma U(s', o)] = \mathbb{E}_{\substack{s', o \sim P \\ a \sim \pi_{o, \theta}}} [-A(s', o) \nabla_{\eta} \beta_{o, \eta}(s')] \end{aligned}$$

Results

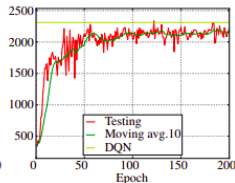


Can learn meaningful actions.

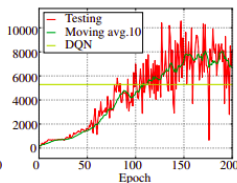
Results



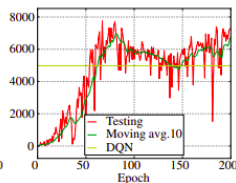
(a) Asterix



(b) Ms. Pacman



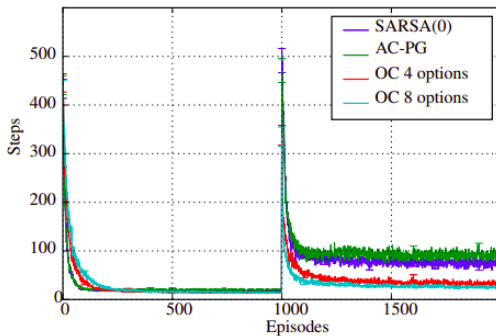
(c) Seaquest



(d) Zaxxon

Comparable performance with DQN

Transfer properties



- ▶ Four rooms environment, trained sequentially on two random tasks.
- ▶ Can adapt better to new situations/goals in the same environment.

Issues

- ▶ **Unstable learning**

- use baselines, use more stable policy optimization methods

- ▶ **Options tend to be deterministic**

- seems like a curse of policy gradient methods, not an O-C method in particular

- add entropy regularizer to the intra-policy gradient

- ▶ **Options tend to disappear**

- termination gradient tends to shrink options duration

- add regularizing constant to the advantage for the stretching effect

- ▶ Assumes whole state space initiation sets

- not an issue, but it's not always a reasonable assumption

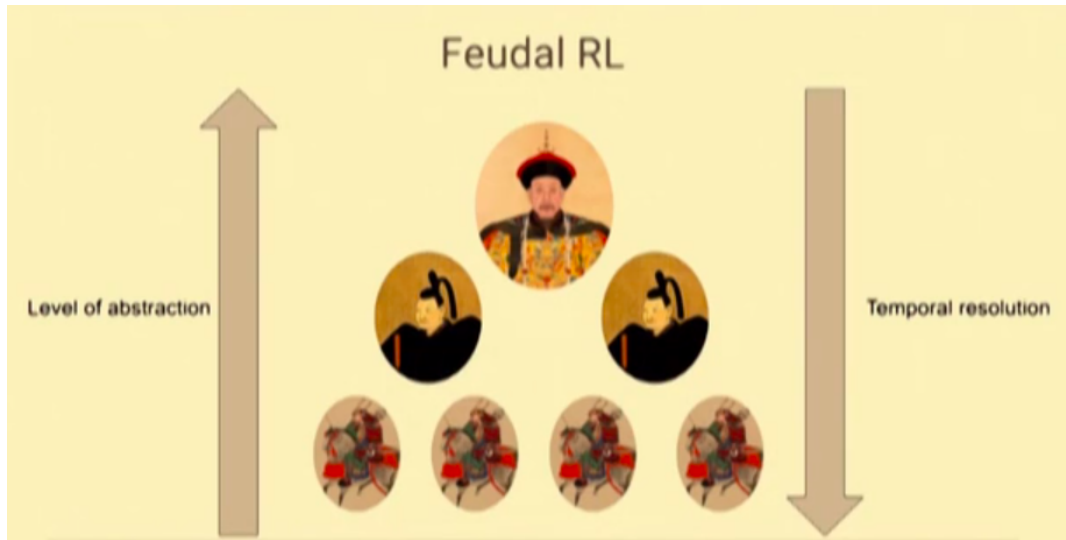
Any alternatives?

Any alternatives to SMDP-based Options Framework approach?

Yes! Feudal RL ³

³Feudal reinforcement learning (1993)

Feudal RL



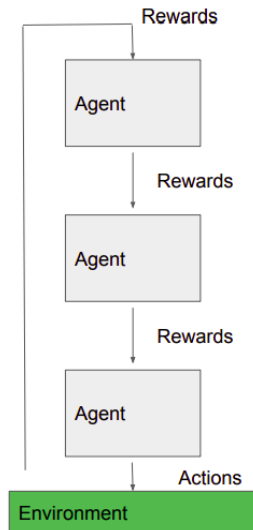
Feudal RL

► Manager

- specifies goals to Worker
- rewards Worker only for reaching goals
- doesn't care about how Worker achieves his goals

► Worker

- just learns to obey Manager



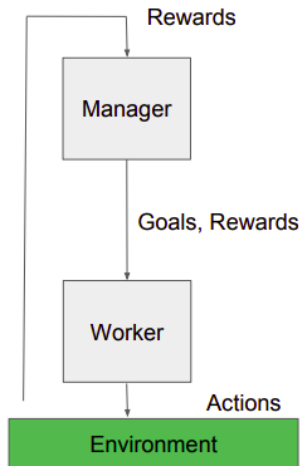
FeUdal Networks for Hierarchical Reinforcement Learning (FuN) ⁴

► Manager

- specifies goals to Worker
- rewards Worker only for reaching goals
- doesn't care about how Worker achieves his goals

► Worker

- just learns to obey Manager



⁴FeUdal Networks for Hierarchical Reinforcement Learning (2017)

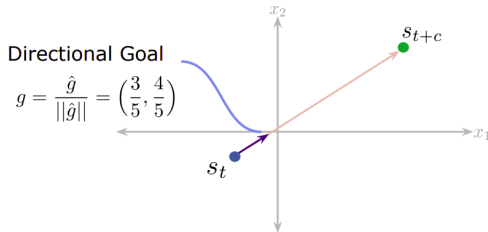
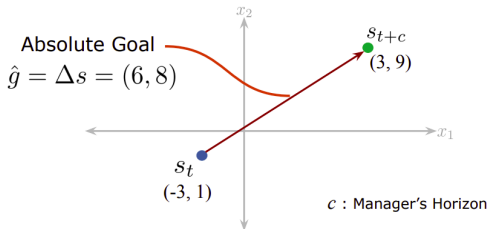
Manager

State: $s_t = f(x_t) \in R^d$

- ▶ operates in his latent state space

Action: $g_t = \mu(s_t) \in R^d$

- ▶ direction in the same latent space
- ▶ set for the time horizon c



How to train manager to lead?
How to train worker to follow?

Manager's transition policy

Manager's policy: $g_t = \mu(s_t)$

- ▶ Defines transition distribution $p(s_{t+c}|s_t, g_t)$
- ▶ Also defines isomorphic MDP with Manager having *transition policy*

$$\pi^{TP}(s_{t+c}|s_t) = p(s_{t+c}|s_t, \mu(s_t))$$

- ▶ Could optimize transition policy with Policy Gradient

$$\nabla_{\Theta} J(\pi_{\Theta}^{TP}) = \mathbb{E}[(R_t - V(s_t)) \nabla \log p(s_{t+c}|s_t, \mu(s_t))]$$

Manager's transition policy

Manager's policy: $g_t = \mu(s_t)$

- ▶ Defines transition distribution $p(s_{t+c}|s_t, g_t)$
- ▶ Also defines isomorphic MDP with Manager having *transition policy*

$$\pi^{TP}(s_{t+c}|s_t) = p(s_{t+c}|s_t, \mu(s_t))$$

- ▶ Could optimize transition policy with Policy Gradient

$$\nabla_{\Theta} J(\pi_{\Theta}^{TP}) = \mathbb{E}[(R_t - V(s_t)) \nabla \log p(s_{t+c}|s_t, \mu(s_t))]$$

Problem

Transition distribution is unknown! (depends on the worker)

Manager's transition policy

Manager's policy: $g_t = \mu(s_t)$

- ▶ Defines transition distribution $p(s_{t+c}|s_t, g_t)$
- ▶ Also defines isomorphic MDP with Manager having *transition policy*

$$\pi^{TP}(s_{t+c}|s_t) = p(s_{t+c}|s_t, \mu(s_t))$$

- ▶ Could optimize transition policy with Policy Gradient

$$\nabla_{\Theta} J(\pi_{\Theta}^{TP}) = \mathbb{E}[(R_t - V(s_t)) \nabla \log p(s_{t+c}|s_t, \mu(s_t))]$$

Problem

Transition distribution is unknown! (depends on the worker)

Trick

- ▶ Make an assumption on transition distribution p
- ▶ Intrinsically motivate the Worker to make that assumption hold

Transition Policy Gradient

Assumption: $p \propto e^{d_{\cos}(s_{t+c}-s_t, g_t)}$

- ▶ von Mises-Fisher distribution
- ▶ Intuition: similar to normal distribution on a unit circle/sphere

Transition Policy Gradient

$$\nabla_{\Theta} J(\pi_{\Theta}^{TP}) = \mathbb{E}[(R_t - V(s_t)) \nabla d_{\cos}(s_{t+c} - s_t, \mu(s_t))]$$

Intrinsic reward

$$r_{t+c}^I = d_{\cos}(s_{t+c} - s_t, g_t)$$

Train worker

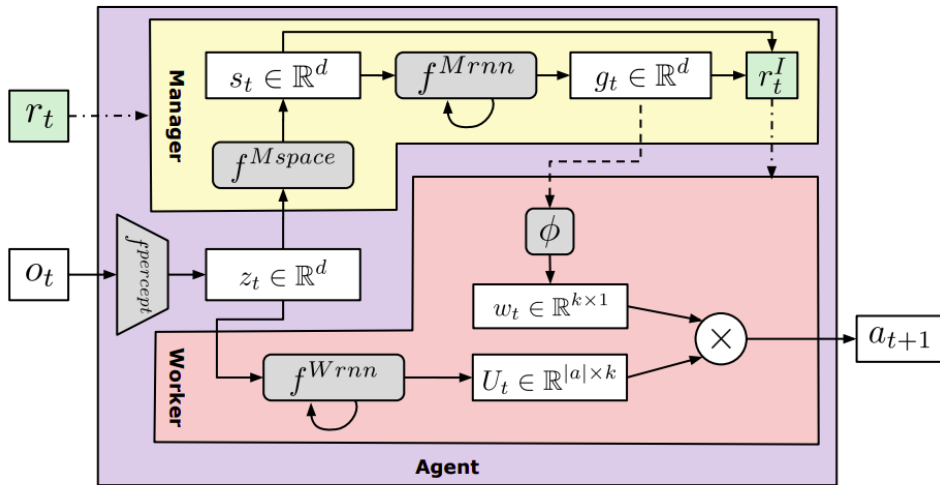
Worker

- ▶ State $s_t^W = (o_t, g_t)$
- ▶ Softmax-policy emits primitive actions a_t
- ▶ Get rewards from worker

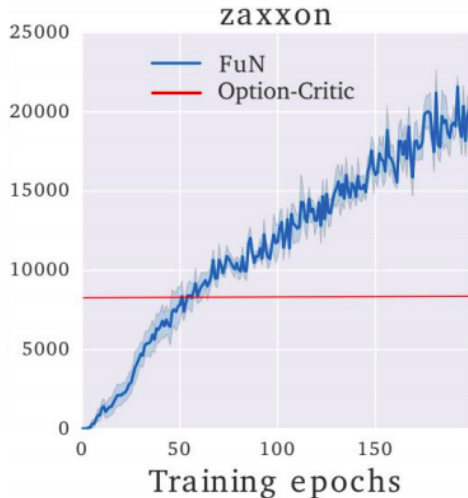
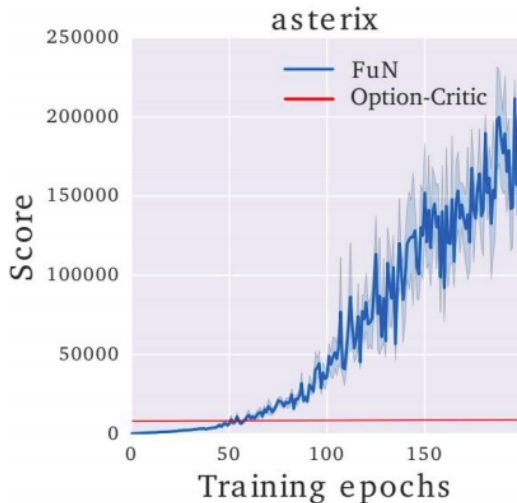
$$r_t = r_t^I + \alpha \cdot r_t^E$$

Train with any policy optimization method.

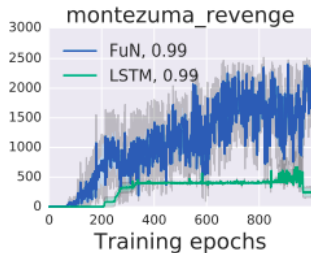
Architecture



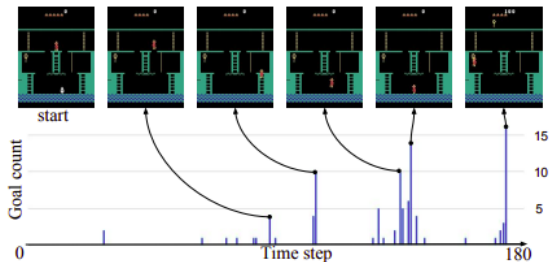
FuN vs Option-Critic



Montezuma's Revenge: FuN vs LSTM



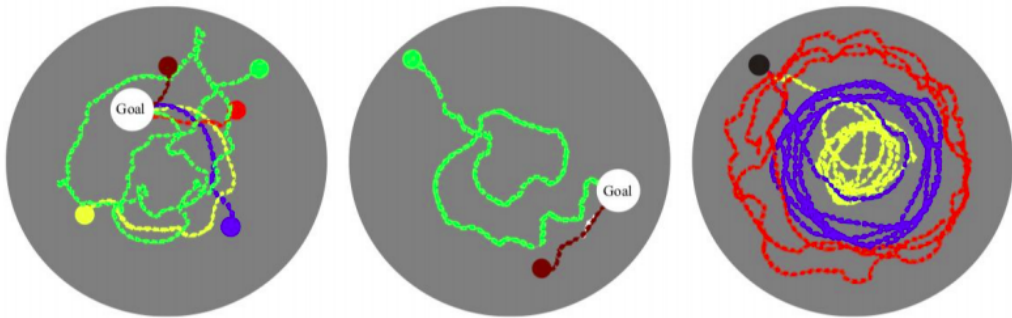
(a)



(b)

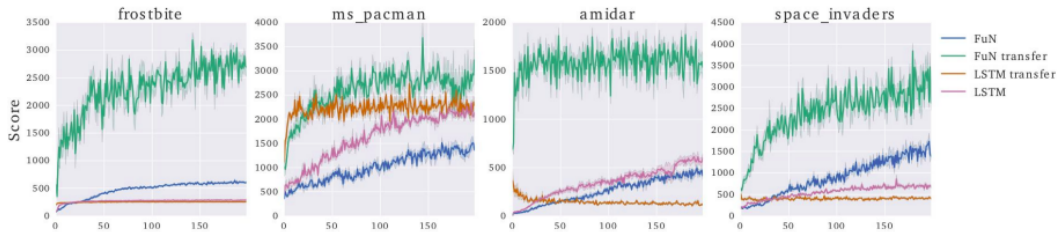
- ▶ Outperforms LSTM baseline
starts learning earlier, requires less intensive hyper-parameter search
- ▶ Learns reasonable goals
- ▶ 400 score corresponds to solving the first room (take the key, open a door)

Qualitative analysis



- ▶ Circular maze with invisible goal
- ▶ Agent is teleported to the random location every time he reaches the goal
- ▶ Left: trajectories for individual episodes
- ▶ Right: colored different goal-induced [meaningful!] trajectories

Atari action repeat transfer



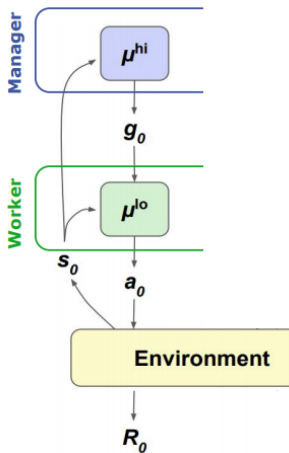
- ▶ One of the goals of HRL - better transfer learning
- ▶ Tested transfer learning with different number of action repeats

HIRO: Hierarchical Reinforcement learning with Off-policy correction ⁵

- ▶ Based on Feudal RL
looks similar to FuN
- ▶ Action space is continuous
continuous control tasks
- ▶ Off-policy
sample efficiency, lesser variance

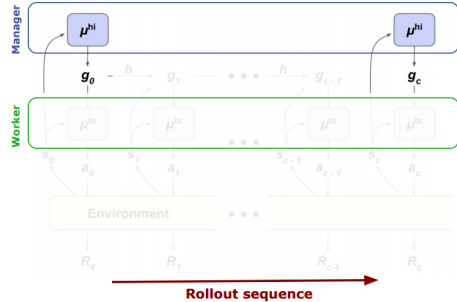
⁵Data-Efficient Hierarchical Reinforcement Learning (2018)

Agent ⁶

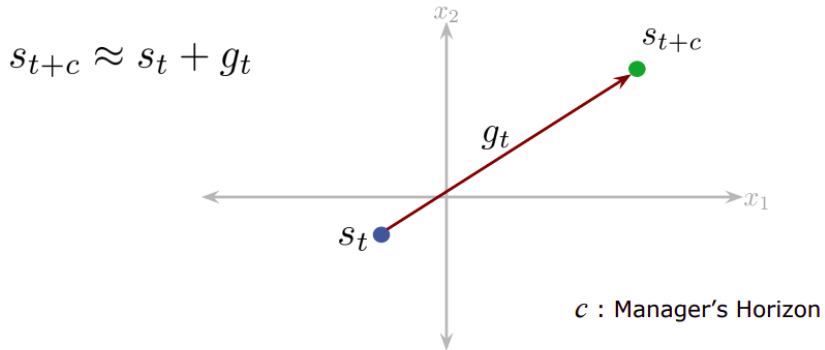


Manager

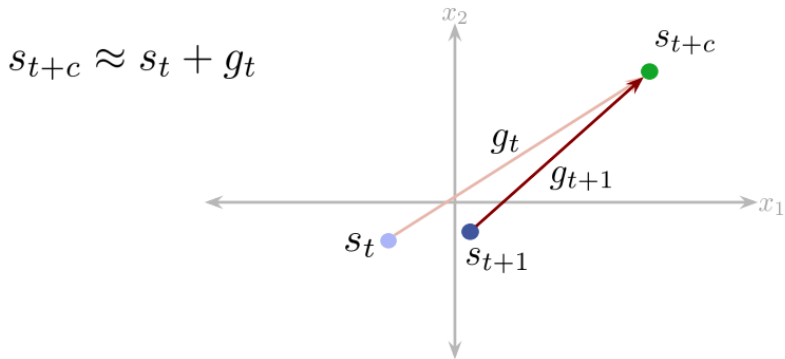
- ▶ Sets goals every c steps
 - ▶ another time scale
- ▶ Operates within the raw observation state space
 - ▶ simplifies things
 - ▶ has its own benefits during learning



Goal representation

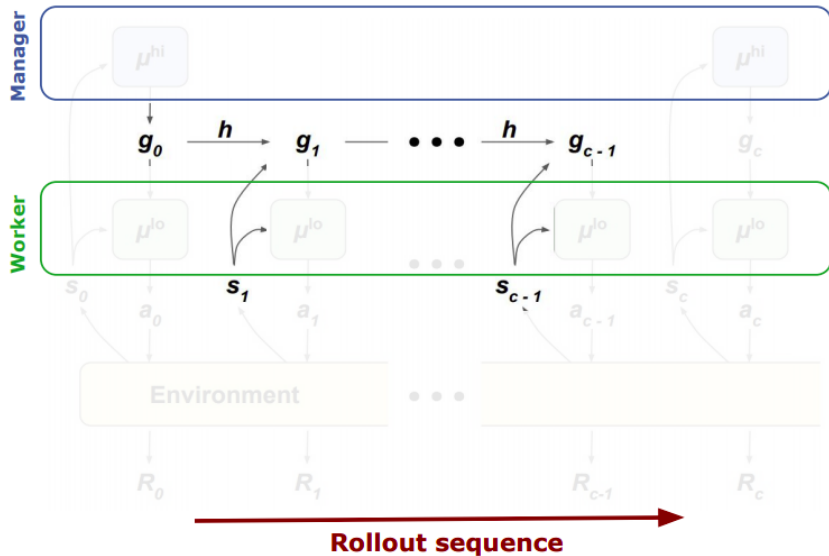


Goal update

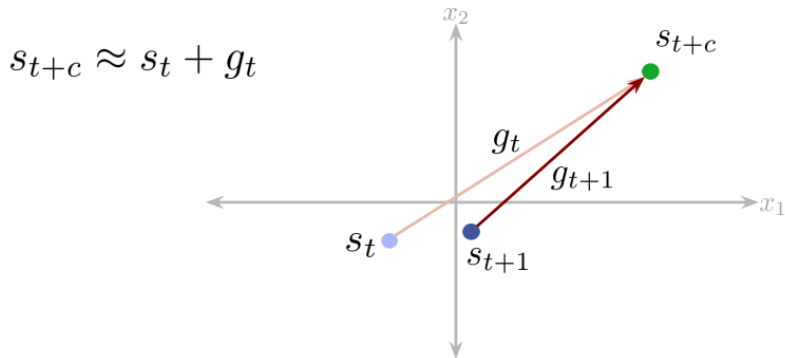


$$g_{t+1} = h(s_t, g_t, s_{t+1}) = s_t + g_t - s_{t+1}$$

Goal update

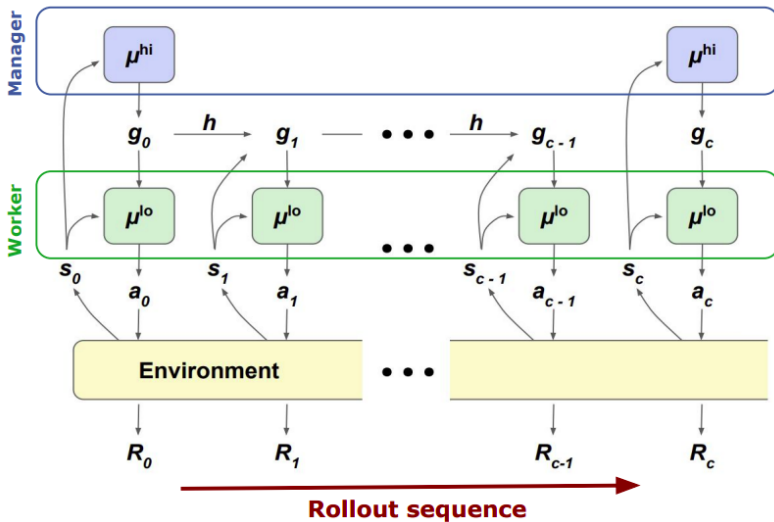


Intrinsic reward

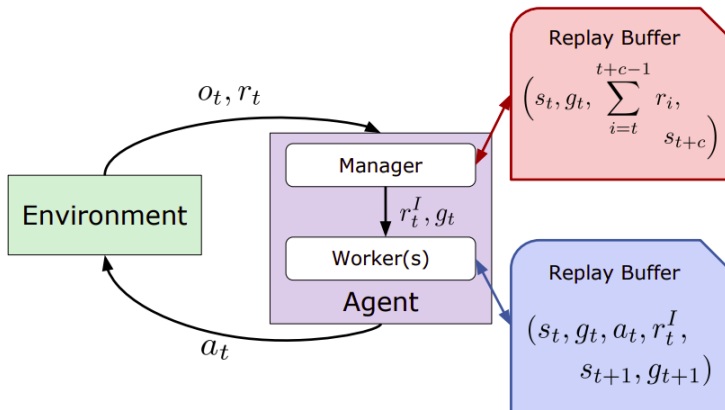


$$r^I(s_t, g_t, a_t, s_{t+1}) = \|s_t + g_t - s_{t+1}\|_2 = \|g_{t+1}\|_2$$

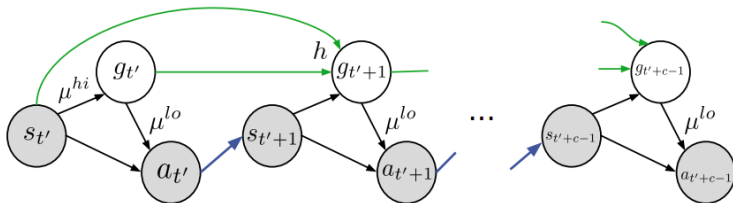
Ready to collect experience



Ready to train?

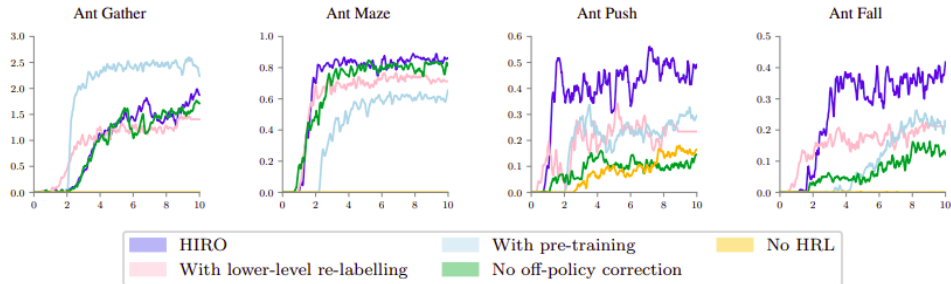
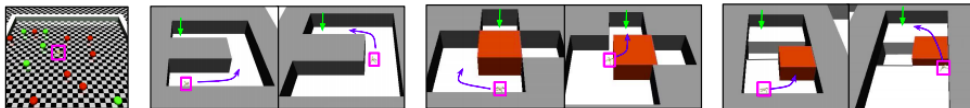


Goal re-labelling

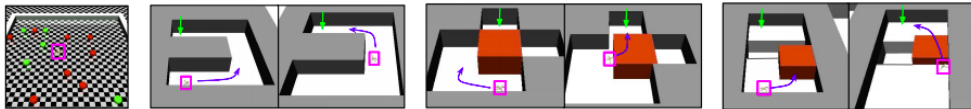


$$\tilde{g}_{t'} = \operatorname{argmax}_{\tilde{g}_{t'}} \mu^{lo}(a_{t':t'+c-1} | s_{t':t'+c-1}, \tilde{g}_{t':t'+c-1})$$

Ablative analysis⁷



HIRO vs FuN⁸



	Ant Gather	Ant Maze	Ant Push	Ant Fall
HIRO	3.02 ± 1.49	0.99 ± 0.01	0.92 ± 0.04	0.66 ± 0.07
FuN representation	0.03 ± 0.01	0.0 ± 0.0	0.0 ± 0.0	0.0 ± 0.0
FuN transition PG	0.41 ± 0.06	0.0 ± 0.0	0.56 ± 0.39	0.01 ± 0.02
FuN cos similarity	0.85 ± 1.17	0.16 ± 0.33	0.06 ± 0.17	0.07 ± 0.22
FuN	0.01 ± 0.01	0.0 ± 0.0	0.0 ± 0.0	0.0 ± 0.0

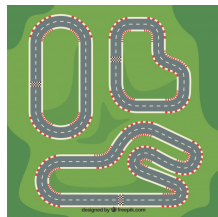
⁸As an open-source implementation of FuN wasn't available at that moment, it was implemented just by adding some of its key properties to HIRO.

Multi-task setting

Until now, only single-task settings were considered for learning.

Intuition: if HRL methods learn transferable skills, then

- ▶ can we learn these skills on different tasks?
- ▶ will it improve learning speed on new unseen task?



Multi-task setting

Consider meta-task in multi-task setting:

- ▶ tasks are sampled from the set of similar tasks
- ▶ learn any task from the set quickly
 - ~ maximize expected return over the lifetime of the agent on one task

HRL goal in this setting:

- ▶ find a set of low-level primitives that enable the high-level policy to be learned quickly

Can we just plug-in Option-Critic?

Actually, yes

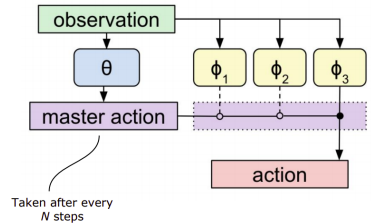
- ▶ Learned options are good primitives to solve different tasks
- ▶ But the master policy is task specific

Meta Learning Shared Hierarchies

Inspired by Options Framework

fixed duration options $\Rightarrow$ option is just a policy
(aka *sub-policy*)

Agent = Master-policy + a set of sub-policies



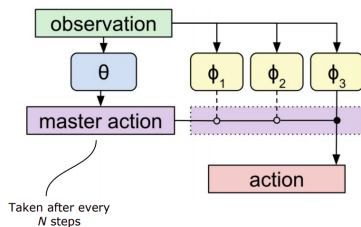
Meta Learning Shared Hierarchies

Master-policy

- ▶ task-specific - new master-policy for every new task
- ▶ operates on another time scale - every N steps

Sub-policies

- ▶ shared between tasks for reuse



MLSH. One-task training

```
Initialize master-policy  
for W steps (warmup) do  
    Collect D steps of experience  
    Update master-policy  
end
```

```
for U steps (joint updates) do  
    Collect D steps of experience  
    Update master-policy  
    Update shared sub-policies  
end
```

- ▶ Every sub-policy is learned only on its own parts of trajectories
- ▶ Master-policy and sub-policies both optimize expected reward over episode
- ▶ Warm up: master-policy learns how to use available primitives

MLSH. Multi-task training

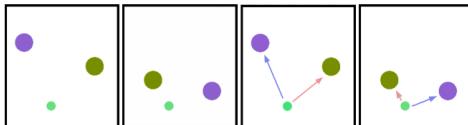
```
Initialize shared sub-policies  
do  
    Sample task from the set  
    Do one-task training  
until ...
```

MLSH. Training

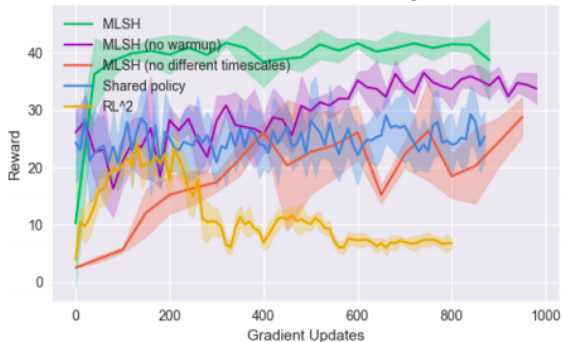
```
Initialize shared sub-policies
do
  Sample task from the set
  Initialize master-policy
  for W steps (warmup) do
    Collect D steps of experience
    Update master-policy
  end

  for U steps (joint updates) do
    Collect D steps of experience
    Update master-policy
    Update shared sub-policies
  end
end
until ...
```

MLSH. Quality analysis ⁹



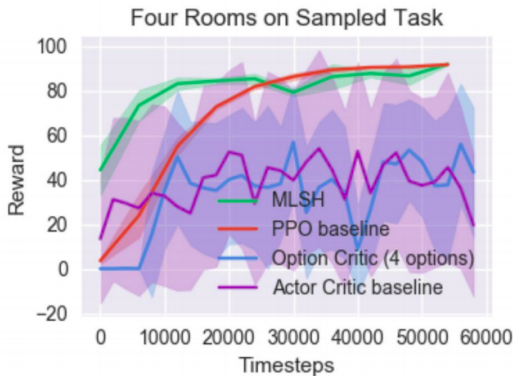
MovementBandits Full Training



MovementBandits on Sampled Task

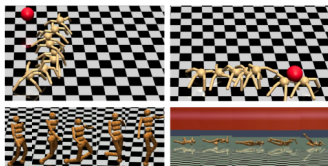


MLSH. Quality analysis



- ▶ Option-Critic performs similarly to its baseline Actor-Critic
- ▶ MLSH reaches high rewards faster than its baseline PPO
- ▶ Comparison MLSH vs Option-Critic is not strictly valid (different baselines)

MLSH. Results ¹⁰ ¹¹

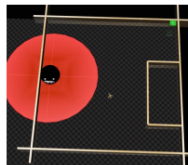


Reward on Walk/Crawl combination task	
MLSH Transfer	14333
Shared Policy Transfer	6055
Single Policy	-643

Mixed tasks require diverse sub-policies

¹⁰Video: Walk/Crawl

¹¹Video: Ant Obstacle



Reward on Ant Obstacle task	
MLSH Transfer	193
Single Policy	0

Unsolvable for non-HRL methods

Summary

What is Hierarchical RL and what are its goals?

- ▶ Make RL agents find and use abstractions at different scales

How to formalize high-level actions and make RL agents use them?

- ▶ Options Framework + Option-Critic

How to make RL agents operate at different scales another way?

- ▶ Feudal RL + FuN/HIRO
- ▶ High-level action is a downward goal between the two layers of the managerial hierarchy

How to apply Hierarchical approach to multi-task setting and is it useful?

- ▶ Options Framework + MLSH
- ▶ Learned abstractions could be reused